WINE QUALITY ANALYSIS USING MACHINE LEARNING

INTERDISCIPLINARY PROJECT

Submitted in partial fulfillment of the requirements for the award of
Bachelor of Engineering degree in Computer Science and Engineering

By

PINDI LOKESH KUMAR (Reg. No – 41110965)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SCHOOL OF COMPUTING

SATHYABAMA

INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)
CATEGORY - 1 UNIVERSITY BY UGC
Accredited "A++" by NAAC I Approved by AICTE
JEPPIAAR NAGAR, RAJIV GANDHI SALAI, CHENNAI
600119

MAY – 2024

SATHYABAMA
INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)
Category - I University by UGC
Accredited "A++" by NAAC | Approved by AICTE
www.sathyabama.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this Project Report is the bonafide work of **PINDI LOKESH KUMAR (41110965)** who carried out the Project entitled "**WINE QUALITY ANALYSIS USING MACHINE LEARNING**" under my supervision from January 2024 to April 2024.

Internal Guide

Ms. M. MADHAVI M. Tech.,

Head of the Department

Dr. L. LAKSHMANAN, M.E., Ph.D.,

Submitted for Interdisciplinary Viva voce Examination held on _____

Internal Examiner **External Examiner**

DECLARATION

I, **PINDI LOKESH KUMAR (Reg. No- 41110965),** here by declare that the Project Report entitled**"WINE QUALITY ANALYSIS USING MACHINE LEARNING"** done by me under the guidance of **Ms.M.MADHAVI M.Tech** Is submitted in partial fulfillment of the requirements for the award of Bachelor of Engineering degree in **Computer Science and Engineering**.

DATE:

PLACE: Chennai **SIGNATURE OF THE CANDIDATE**

ACKNOWLEDGEMENT

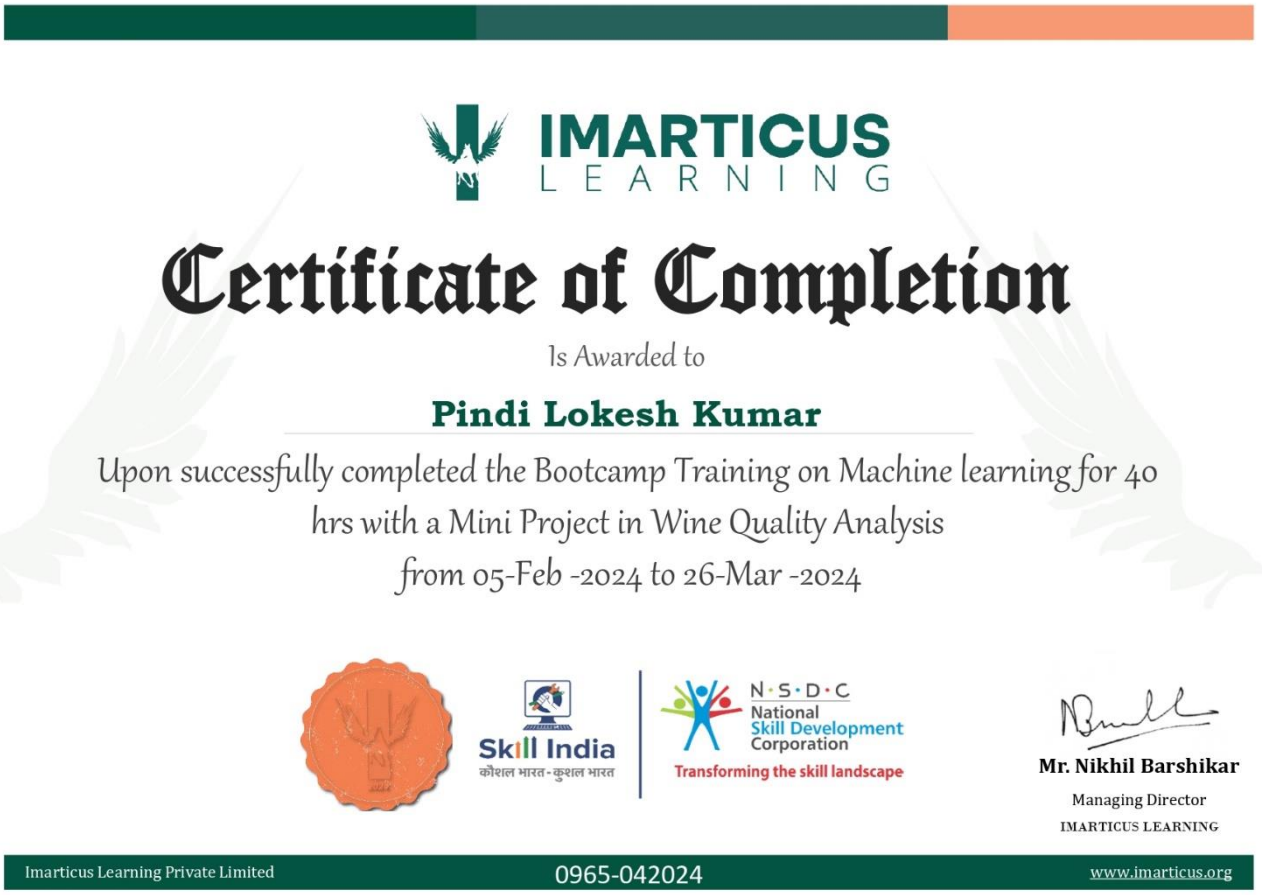
I am pleased to acknowledge my sincere thanks to **Board of Management** of **SATHYABAMA** for their kind encouragement in doing this project and for completing it successfully. I am grateful to them.

I convey my thanks to **Dr. T.Sasikala M.E., Ph.D.**, **Dean**, School of Computing ,and **Dr. L. Lakshmanan, M.E., Ph.D., Head of the Department of Computer Science and Engineering** for providing me necessary support and details at the right time during the progressive reviews.

I would like to express my sincere and deep sense of gratitude to my Project Guide **Ms.M.MADHAVI M.Tech** for her valuable guidance, suggestions and constant encouragement which paved way for the successful completion ofmy phase-1 professional Training.

I wish to express my thanks to all Teaching and Non-teaching staff members of the **Department of Computer Science and Engineering** who were helpful in many ways for the completion of the project.

CERTIFICATE



ABSTRACT

The quality of a wine is important for the consumers as well as the wine industry. The traditional way of measuring the quality of wine is very difficult and time-consuming. Nowadays, machine learning models are important tools that are used to replace human tasks. In this case, there are several features to predict the wine quality but the entire features will not be relevant for better prediction.This project aims to unravel the intricate relationships between physicochemical attributes and sensory perception, offering valuable insights into wine quality assessment and production refinement. By leveraging advanced data analysis and machine learning techniques, we aspire to empower the wine industry with a deeper understanding of winequality and contribute to the continuous pursuit of excellence in winemaking. The resultsof this project will provide a valuable framework for wine quality assessment andrefinement in the wine indust .

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	CERTIFICATE	v
	ABSTRACT	vi
	LIST OF FIGURES	viii
1	**INTRODUCTION**	
	1.1 Overview	1
	1.2 Dataset description	3
2	**LITERATURE SURVEY**	
	2.1 Survey	5
3	**REQUIREMENTS ANALYSIS**	
	3.1 Objective	7
	3.2 Software Requirements	7
4	**DESIGN DESCRIPTION OF PROPOSED PRODUCT**	
	4.1 Proposed Product	9
	4.1.1 Ideation Map/Architecture Diagram	9
	4.1.2 Various stages	10
	4.1.3 Internal or Component design structure	27
5	**CONCLUSION**	28
6	**REFERENCES**	29

LIST OF FIGURES

Figure No.	Figure Name	Page No.
4.1.1	System architecture	9
4.2.1	Using head() function	11
4.3.1	Using info() function	12
4.3.2	Using describe() function	13
4.3.3	Correlation matrix	14
4.3.4	Importance plot	15
4.3.5	Plot of alcohol and target variable quality	15
4.4.1	Boxplot	17
4.4.2	Pairplot	17
4.4.3	Boxplot after handling outliers	18
4.4.4	Missing values	19
4.4.5	After handling missing values	20
4.5.1	Counting instances	21
4.5.2	Classification version of target variable	21
4.5.3	Balancing dataset	22

4.5.4	Correlation between columns	23
4.6.1	Normalizing the data	24
4.7.1	Confusion matrix	26
4.7.2	Model accuracy	26
4.7.3	Model precision	27
4.7.4	Internal design structure	27

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW

The wine industry, a significant player in the global market, constantly seeks ways to improve the quality of its products to cater to the diverse preferences of consumers. This project aims to analyse the quality of wines using a dataset comprising physicochemical attributes and sensory data. The dataset contains 11 input variables, such as fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulphur dioxide, total sulphur dioxide, density, pH, sulphates, and alcohol. The output variable represents the quality of wines and is rated on a scale from 0 to 10.

The first phase of the project involves data pre-processing, where two datasets from different sources are combined, and missing values are handled. Due to data collection issues, some values were randomly removed, necessitating careful imputation to ensure the integrity of the dataset. Outlier detection algorithms will also be utilized to identify potential anomalies in the data, specifically aiming to detect exceptional or poor wines that deviate significantly from the general trend. These outliers can significantly impact the model's performance and provide valuable insights into the factors affecting wine quality.

The unbalanced nature of the dataset poses a challenge, as there are substantially more samples of normal wines compared to excellent or poor ones. To address this issue, various strategies will be employed, such as oversampling, under sampling, or class weighting, to create a balanced dataset suitable for training and evaluation.

A critical aspect of this project involves assessing the relevance of the input variables for predicting wine quality. Not all attributes may have a significant impact on the output, and some may even introduce noise to the model. Feature selection methods, such as correlation analysis, recursive feature elimination, or LASSO regularization, will beapplied to identify the most informative and influential attributes. The objective is to create a

compact and accurate model that captures the essence of wine quality without unnecessary complexity. The main analysis revolves around building classification anregression models to predict wine quality based on the selected attributes. Various machine learning algorithms, including Random Forest, Support Vector Machines, and Gradient Boosting, will be explored to identify the best-performing model. The models will be trained on a portion of the data and evaluated using performance metrics such as accuracy, precision, recall, and F1-score. The evaluation will be conducted using k-fold cross-validation to ensure robustness and generalization of the model.

To visualize the relationships between the attributes and wine quality, exploratory data analysis will be performed. Pair plots, correlation matrices, and scatter plots will provide insights into the dependencies between variables, potential collinearity issues, and potential nonlinear relationships.

The results of this project will provide a valuable framework for wine quality assessment and refinement in the wine industry. Accurate prediction models can assist winemakers in understanding the factors that contribute to wine quality and making informed decisions to enhance product quality.

1.2 DATASET DESCRIPTION:

These attributes contain eleven distinct variables that are essential aspects of wine composition and characteristics:

Input Variables - Physicochemical Attributes.

Fixed Acidity:

Fixed acidity refers to the concentration of non-volatile acids present in the wine, which contributes to its overall tartness and acidity level.

Volatile Acidity:

Volatile acidity is given in this column and indicates the presence of volatile acids in the wine. High levels of volatile acidity can lead to undesirable flavours and aromas, often associated with vinegar-like characteristics.

Citric Acid:

Citric acid content is recorded in this column. Citric acid, a weak organic acid, can contribute to the wine's freshness and enhance its overall flavour profile.

Residual Sugar:

The residual sugar column quantifies the amount of sugar left in the wine after fermentation is complete. It contributes to the wine's sweetness and can influence its perceived balance.

Chlorides:

Chloride levels can impact the wine's taste, with higher levels potentially leading to a salty or bitter taste.

Free Sulphur Dioxide:

Sulphur dioxide is used as a preservative and antioxidant in winemaking, and its levels can affect both the wine's stability and aroma.

Total sulphur Dioxide:

Total sulphur dioxide includes both free and bound sulphur dioxide. It provides a comprehensive measure of the wine's sulphur content, impacting its preservation and sensory characteristics.

Density:

Density is influenced by factors such as sugar content and alcohol levels and can provide insights into the wine's body and mouthfeel.

pH:

The pH column represents the level of acidity or alkalinity in the wine. pH influences various chemical reactions and can affect the wine's stability and sensory properties.

Sulphates:

Sulphates can act as antioxidants and antimicrobial agents, contributing to the wine's longevity and overall quality.

Alcohol:

The alcohol level in wine contributes to its body, texture, and perceived warmth when consumed.

Output Variable - Quality:

Quality:

The quality column represents the output variable, indicating the sensory based quality assessment of the wines on a scale of 0 to 10. This score reflects the overall perception of the wine's quality by experts or consumers.

Each of these columns plays an important role in creating the composition, characteristics, and ultimately, the quality of the wines. This project aims to understand the relationships between these attributes and the sensory-based quality assessment to build a reliable predictive model.

CHAPTER 2

LITERATURE REVIEW

2.1 SURVEY

Rohan Kothawade (2021)

The main objective of this research was done to speed up the process of wine quality testing and reduce the time and effort put in by human wine testing experts. The Machine Learning methods used in this paper are Artificial Neural Network (ANN), Support Vector Machine (SVM), Naïve-Bayes (NB). The prediction results obtained by Artificial NeuralNetwork algorithm was better than the results of the Support Vector Machine algorithm and theNaïve-Bayes algorithm for both red wine and white wine datasets.

Lee et al., (2015)

Lee had proposed a decision tree-based method to predict the wine quality and compare their approach using three machine learning algorithm such as support vector machine, multi-layer perceptron, and BayesNet. They found their proposed method is better compared to other stated methods.

Sachin Boithe (2019)

This project focused on making the difficult task of wine quality analysis which is only known by experts to be available to common the people without consuming as much time. The methods such as Random Forest, Stochastic Descent, Support Vector Classification and Logistic Regression were used in this certain paper. Various parameters such as Acidity (including Citric acid), Chlorides, Sulphur Dioxide, Density, pH, Alcohol Content were used to find the quality.

Maximum accuracy was achieved in this project was by Random Forest at 88%

followed by Logistic Regression at 86% then Support Vector Classification at 85% while the least accurate was Stochastic Gradient at 81%.

P. Appalasamy et al., (2012)

He had predicted the wine quality based on the physiochemical data. Both red wine and white wine datasets were used, and the decision tree and naive Bayes algorithms were applied. The results of these two algorithms were compared and it was concluded that the classification approach can help to improve the wine quality during production.

CHAPTER 3

REQUIREMENTS ANALYSIS

3.1 OBJECTIVE OF THE PROJECT

The traditional way of measuring the quality of wine is time consuming.

The main objective of this project is to build a robust predictive model to accurately assess wine quality based on physicochemical attributes and sensory data

3.2 SOFTWARE REQUIREMENTS:

Various libraries such as the following are required to perform this analysis

NumPy :

NumPy is the fundamental package for scientific computing in Python. It is a Python library that provides a multidimensional array object, various derived objects (such as masked arrays and matrices), and an assortment of routines for fast operations on arrays, including mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation and much more.

Pandas :

Pandas is a Python library used for working with data sets. It has functions for analyzing, cleaning, exploring, and manipulating data. It is mainly used for data analysis and associated manipulation of tabular data in data frames.

Seaborn :

Seaborn is a library for making statistical graphics in Python. It builds on top of matplotlib and integrates closely with pandas data structures. Seaborn helps you

explore and understand your data.

Matplotlib :

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. It is a plotting library for the Python programming language and its numerical mathematics extension NumPy. It provides an object-oriented API for embedding plots into applications using general-purpose GUI toolkits .

Sklearn :

Scikit-learn (Sklearn) is the most useful and robust library for machine learning in Python. It provides a selection of efficient tools for machine learning and statistical modelling including classification, regression, clustering and dimensionality reduction via a consistence interface in Python.

sklearn.model_selection is used to split our data into train and test sets where feature variables are given as input in the method. test_size determines the portion of the data which will go into test sets and a random state is used for data reproducibility.

sklearn. preprocessing package provides several common utility functions and transformer classes to change raw feature vectors into a representation that is more suitable for the downstream estimators.

sklearn. ensemble module includes two averaging algorithms based on randomized decision trees: the Random Forest algorithm and the Extra-Trees method.

sklearn. metrics module implements several loss, score, and utility functions to measure classification performance. Some metrics might require probability estimates of the positive class, confidence values, or binary decisions values.

CHAPTER 4

DESIGN DESCRIPTION OF PROPOSED PROJECT

4.1 PROPOSED METHODOLOGY:

In this system we use a machine learning algorithm namely random forest to determine the quality of wine.

Initially exploratory data analysis is performed on the given dataset followed by pre-processing of the data. The data is further divided into training (70%) and testing (30%) sets, with the training set being utilised to train the model utilising Random Forest algorithm. The testing set is used to determine the accuracy and the precision of the model. The trained model is used to determine the testing set's correctness. The accuracy of the algorithm is found along with its precision of the algorithm for predicting wine quality.

4.1.1 Ideation Map/System Architecture:

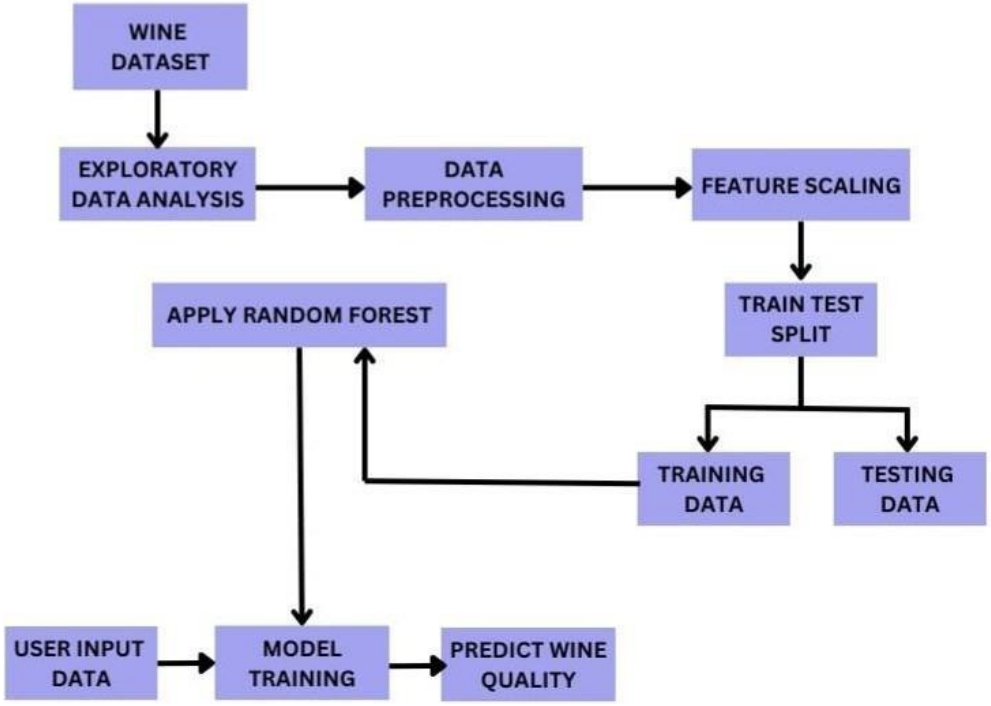


Figure-1: system architecture

4.1.2 Various Stages:

This project is done in the following steps

STEP 4.1: Importing Libraries

Import the required libraries .

For wine quality analysis the following libraries are used:

NumPy
Pandas
Seaborn
Matplotlib
Sklearn

CODE:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib. pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

STEP 4.2: Loading The Dataset

Read the wine dataset file using pandas' read_excel() function.

CODE:

```
#reading data
data  = pd.read_excel("wine.xlsx")
print(data.head())
```

OUTPUT:

```
   fixed acidity  volatile acidity  citric acid  residual sugar  chlorides  \
0            7.4             0.70         0.00             1.9      0.076
1            7.8             0.88         0.00             2.6      0.098
2            7.8             0.76         0.04             2.3      0.092
3           11.2             0.28         0.56             1.9      0.075
4            7.4             0.70         0.00             1.9      0.076

   free sulfur dioxide  total sulfur dioxide  density    pH  sulphates  \
0                 11.0                  34.0   0.9978  3.51       0.56
1                 25.0                  67.0   0.9968  3.20       0.68
2                 15.0                  54.0   0.9970  3.26       0.65
3                 17.0                  60.0   0.9980  3.16       0.58
4                 11.0                  34.0   0.9978  3.51       0.56

   alcohol  quality
0      9.4      5.0
1      9.8      5.0
2      9.8      5.0
3      9.8      6.0
4      9.4      5.0
```

Figure-2: Using head() function

STEP 4.3: Exploratory Data Analysis

It refers to the process of visually and statistically examining the dataset to uncover patterns, relationships, anomalies, and potential insights. EDA is an essential preliminary step that helps you better in understanding the data's structure, distribution, and characteristics before applying advanced analytical techniques.

4.3.1 Summary of the Dataset

Summary of the dataset can be displayed using the “info()” function in python. It prints the various information of the dataframe such as index type, dtype, columns, non-values, and memory usage. It gives a quick overview of the dataset.

CODE:

```
print(data.info())
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1599 entries, 0 to 1598
Data columns (total 12 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   fixed acidity         1599 non-null   float64
 1   volatile acidity      1599 non-null   float64
 2   citric acid           1599 non-null   float64
 3   residual sugar        1599 non-null   float64
 4   chlorides             1599 non-null   float64
 5   free sulfur dioxide   1599 non-null   float64
 6   total sulfur dioxide  1598 non-null   float64
 7   density               1599 non-null   float64
 8   pH                    1598 non-null   float64
 9   sulphates             1599 non-null   float64
 10  alcohol               1599 non-null   float64
 11  quality               1598 non-null   float64
dtypes: float64(12)
memory usage: 150.0 KB
None
```

Figure-3: Using info() function

4.3.2 Describing Dataset

The describe() function computes a summary of statistics pertaining to the Data Frame columns. If the DataFrame contains numerical data, the descriptioncontains these information for each column:

count - The number of not-empty values.mean

- The average (mean) value.

std - The standard deviation.min

- the minimum value.

25% - The 25% percentile.

50% - The 50% percentile.

75% - The 75% percentile.max

- the maximum value.

CODE:

```
print(data.describe())
```

OUTPUT:

```
       fixed acidity  volatile acidity  citric acid  residual sugar  \
count    1599.000000       1599.000000  1599.000000     1599.000000
mean        8.319637          0.527821     0.270976        2.538806
std         1.741096          0.179060     0.194801        1.409928
min         4.600000          0.120000     0.000000        0.900000
25%         7.100000          0.390000     0.090000        1.900000
50%         7.900000          0.520000     0.260000        2.200000
75%         9.200000          0.640000     0.420000        2.600000
max        15.900000          1.580000     1.000000       15.500000

         chlorides  free sulfur dioxide  total sulfur dioxide      density  \
count  1599.000000          1599.000000           1598.000000  1599.000000
mean      0.087467            15.874922             46.433041     0.996747
std       0.047065            10.460157             32.876249     0.001887
min       0.012000             1.000000              6.000000     0.990070
25%       0.070000             7.000000             22.000000     0.995600
50%       0.079000            14.000000             38.000000     0.996750
75%       0.090000            21.000000             62.000000     0.997835
max       0.611000            72.000000            289.000000     1.003690

                pH    sulphates      alcohol      quality
count  1598.000000  1599.000000  1599.000000  1598.000000
mean      3.498586     0.658149    10.422983     5.636421
std       0.080346     0.169507     1.065668     0.807665
min       2.740000     0.330000     8.400000     3.000000
25%       3.520000     0.550000     9.500000     5.000000
50%       3.520000     0.620000    10.200000     6.000000
75%       3.520000     0.730000    11.100000     6.000000
max       3.900000     2.000000    14.900000     8.000000
```

Figure-4: Using describe() function

4.3.3 Correlation Matrix

Correlation matrix is a mathematical representation of the relationships between multiple variables in a dataset. It provides insights into how pairs of variables are related or co-vary with each other.

Correlation matrices are useful for identifying relationships, feature selection, multi-collinearity, visualisations, model building. A correlation matrix is generated using the `corr()` function.

CODE:

```
#correlation matrix
import matplotlib.pyplot as plt
corr=data.corr()
plt.subplots(figsize=(15,10))
sns.heatmap(corr,xticklabels=corr.columns,yticklabels=corr.columns,annot=True, cmap=sns.diverging_palette(220,20))
plt.show()
```

OUTPUT:

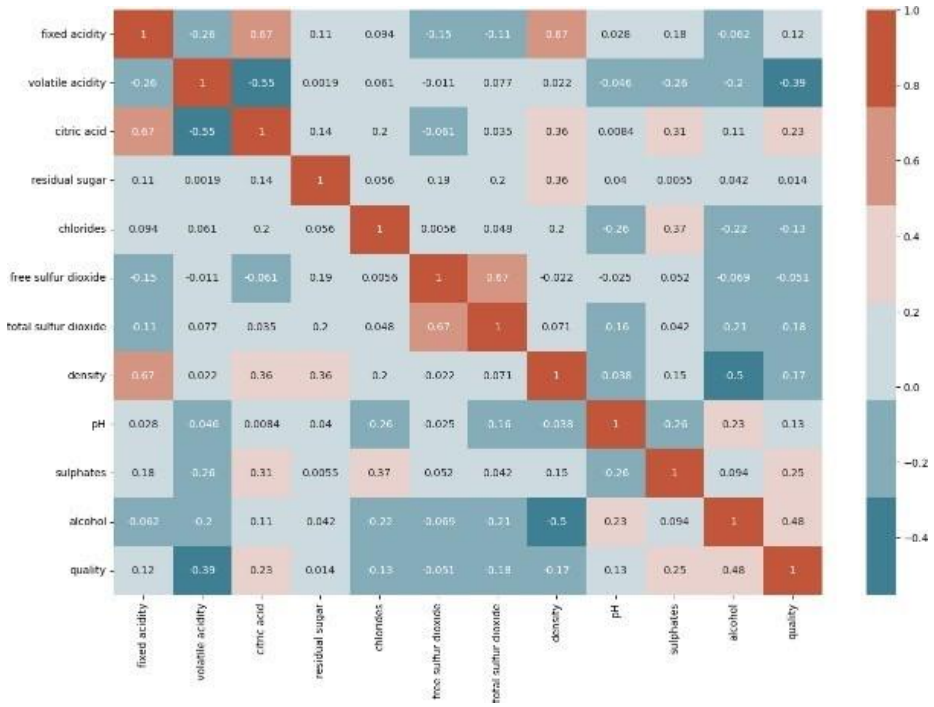


Figure-5: Correlation matrix

4.3.4 Feature Importance With Respect To Target Variable

Showing feature importance with respect to the target variable involves determining which features (input variables) in your dataset have the most significant impact on predicting the target variable (output variable). This process is essential for understanding which attributes contribute the most to the outcome you're trying to predict.

CODE:

```
# Plot Showing Importance of each  feature in the dataset
feat_importances = pd.Series(model.feature_importances_, index=X_features.columns)
feat_importances.nlargest(25).plot(kind='barh',figsize=(10,10))
plt.show()
```

OUTPUT:

From the bellow plot, we can infer that alcohol has a high effect on target variable," quality" when compared to other features in the dataset.

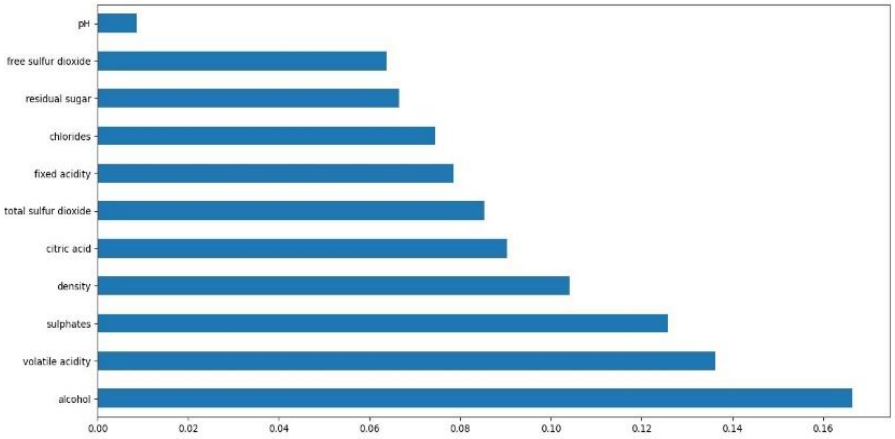


Figure-6: Importance plot

Now, we plot the bar graph in which we check what value of alcohol can be able to make changes in quality.

CODE:

```
#plot of alcohol and target variable quality
plt.figure(figsize=[10,6])
plt.bar(data['quality'],data['alcohol'],color='red')
plt.xlabel("QUALITY")
plt.ylabel("ALCOHOL")
plt.show()
```

OUTPUT:

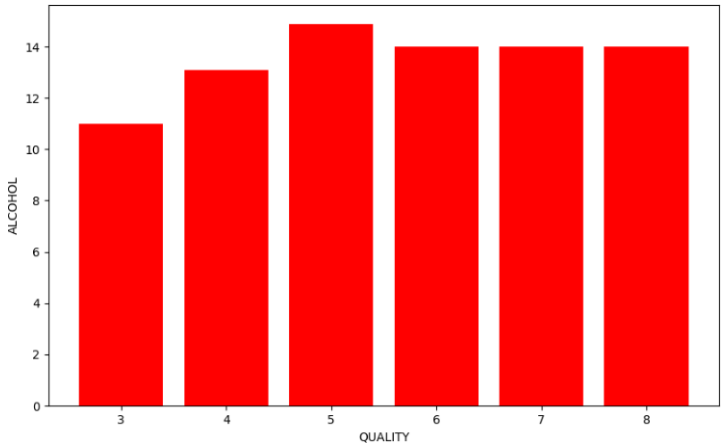


Figure-7: Plot of alcohol and target variable quality

STEP 4.4: Data Pre-processing

Depending on the dataset's characteristics, you might need to perform data preprocessing steps such as handling missing values, dealing with outliers, encoding categorical variables, and scaling numerical features. These steps are crucial to ensure that the data is in a suitable format for analysis and modelling. They are:

Outlier detection and handling them.
Filling missing values.

4.1 Outlier Detection & Analysis

An outlier may be defined as a piece of data or observation that deviates drastically from the given norm or average of the data set. Outlier detection is the process of detecting and subsequently excluding outliers from a given set of data.

4.1.1 Using Boxplot:

Box plots provide a quick visual summary of the variability of values in a dataset. They show the median, upper and lower quartiles, minimum and maximum values, and any outliers in the dataset.

Box plots are useful as they show outliers within a data set. When reviewing a box plot, an outlier is defined as a data point that is located outside the whiskers of the box plot.

CODE:

```
#Data pre-processing :
#displaying outliers
#using boxplot
columns_to_plot = data.columns
plt.figure(figsize=(12, 8))
for p, col in enumerate(columns_to_plot):
    plt.subplot(3, 4, p + 1)
    plt.boxplot(data[col])
    plt.title(col)
    plt.tight_layout()
plt.show()
```

OUTPUT:

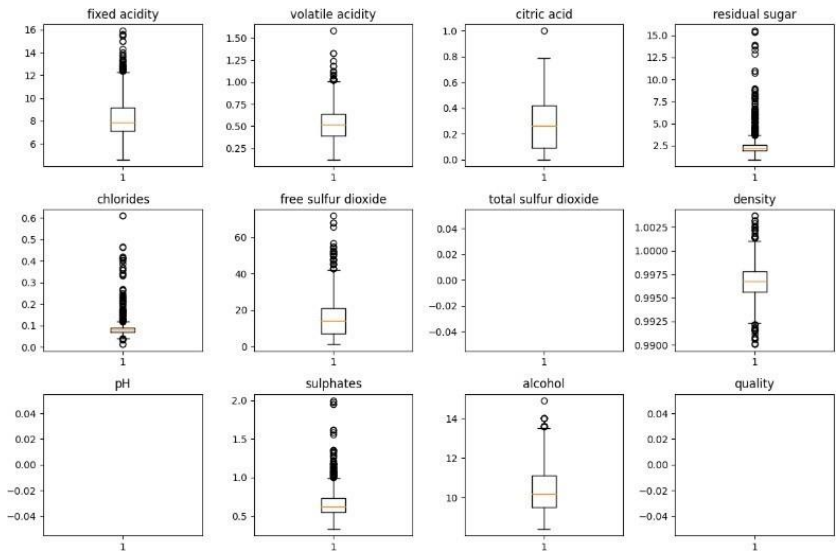


Figure-8: Boxplot

4.4.2 Using Pairplot

A pair plot is a powerful visualization tool that can help in outlier detection, especially in the context of exploring relationships between multiple variables in a dataset. It allows you to create scatter plots for all possible combinations of numerical variables and histograms for the individual variables along the diagonal.

CODE:

```
#using pair plot
columns_to_plot = data.columns
sns.pairplot(data[columns_to_plot])
plt.show()
```

OUTPUT:

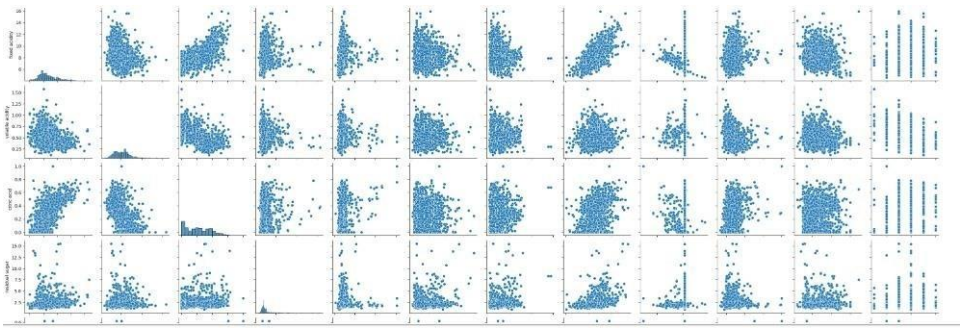


Figure-9: Paiplot

4.4.3 Outlier Analysis And Handling

Outliers can be detected and refilled using the Interquartile Range (IQR) method. IQR is a simple and effective method for detecting outliers and filling missing data, the choice of imputation technique and outlier handling should be based on the specific characteristics of the data.

CODE:

```
#outlier handling and refilling
def refill_outliers_iqr(df,column,lower_bound=0.25,upper_bound=0.75):
  q1 = df[column].quantile(lower_bound)
  q3 = df[column].quantile(upper_bound)
  IQR = q3 - q1
  (parameter) upper_bound: Any
  upper_bound = q3 + 1.5 * IQR
  #replace outliers with nan
  df.loc[(df[column] < lower_bound) | (df[column] > upper_bound), column] = np.nan
  #fill nan with median value
  df[column].fillna(df[column].median(), inplace=True)
  return df
#list columns to detect and refill outliers
columns_to_refill_outliers = data.columns
#detect and refill outliers for each column
for col in columns_to_refill_outliers:
  data = refill_outliers_iqr(data, col)
```

OUTPUT:

The following boxplot figure shows that outliers are successfully removed from the wine dataset.

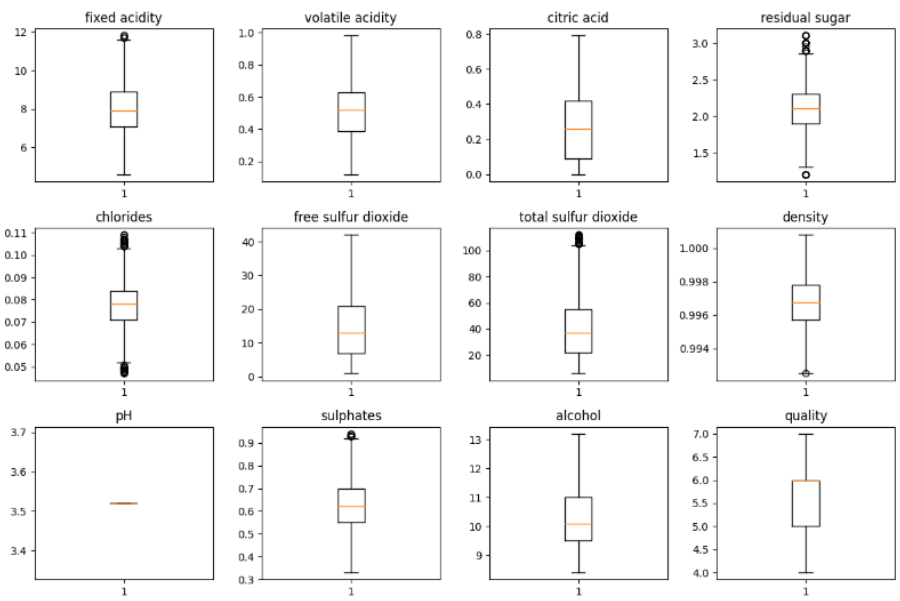


Figure-10 Boxplot after handling outliers

4.4.4 Filling Missing Values

Missing values in a dataset refer to the absence of data for specific observations or attributes. They are represented by null or NaN. Filling missing values is crucial in a dataset because:

- Missing values can lead to biased analyses and inaccurate results.
- Complete data ensures better model performance and generalization.

There are various methods to fill missing values. Few of them are: Mean/Median Imputation, Forward/Backward Fill, Interpolation, regression imputation. Based on the nature of data and missing data patterns, we are filling missing values with the mean value of respective column.

CODE:

```
#filling missing values
print("missing values")
print(data.isnull().sum())
data.update(data.fillna(data.mean()))
print()
print("after filling missing values")
print(data.isnull().sum())
```

OUTPUT:

Displaying missing values of wine dataset:

```
missing values:

fixed acidity           0
volatile acidity        0
citric acid             0
residual sugar          0
chlorides               0
free sulfur dioxide     0
total sulfur dioxide    1
density                 0
pH                      1
sulphates               0
alcohol                 0
quality                 0
dtype: int64
```

Figure-11: Missing values

Displaying after filling missing values of wine dataset:

All columns are showing “0” indicating that there are no missing values in all columns.

```
after filling missing values
fixed acidity           0
volatile acidity        0
citric acid             0
residual sugar          0
chlorides               0
free sulfur dioxide     0
total sulfur dioxide    0
density                 0
pH                      0
sulphates               0
alcohol                 0
quality                 0
dtype: int64
```

Figure-12: After handling missing values

STEP 4.5: Balancing Two Classes

Balancing the two classes refers to addressing the data imbalance issue between different wine quality levels. In your dataset, there might be a significant difference in the number of samples for each quality class (e.g., normal, excellent, poor), which can lead to biased model predictions. Balancing the classes involves ensuring that each quality level has a roughly equal representation to prevent the model from being skewed towards the majority class.

4.5.1 Counting No Of Instances Of Each Class

Counting the number of instances of each class before changing the target variable involves assessing the class distribution in your dataset. This step is crucial to understanding the initial class imbalance and making informed decisions regarding data pre-processing and modelling.

CODE:

```
#counting a number of instances of each class before changing the target variable categorical
data['quality'].value_counts()
```

OUTPUT:

```
5.0    680
6.0    667
7.0    199
4.0     53
Name: quality, dtype: int64
```

Figure-13: Counting instances

4.5.2 Creating A Classification Version Of The Target Variable

This Step involves transforming the continuous quality scores into discrete classes. This step is essential when you want to treat the wine quality assessment as a classification problem with distinct quality levels.

CODE:

Here we are cutting bins use_pd.cut()_in 2 categories 1-5 as BAD and 5-9 as GOOD.

```
from sklearn.preprocessing import LabelEncoder
#create a classification version of table variable
#make 2 categories of target variable
ranges = (1,5,9)
groups = ['bad','good']
data['quality'] = pd.cut(data['quality'],bins=ranges,labels=groups)
#alloting 0 to bad and 1 to good
le = LabelEncoder()
data['quality'] = le.fit_transform(data['quality'])
print("after transforming")
print()
data.head()
```

OUTPUT:

```
after transforming
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.52	0.56	9.4	0
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.52	0.68	9.8	0
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.52	0.65	9.8	0
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.52	0.58	9.8	1
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.52	0.56	9.4	0

Figure-14: Classification version of target variable

After transforming the target variable, we are printing the counts:

CODE:

```
data['quality'].value_counts()
```

```
1    866
0    733
Name: quality, dtype: int64
```

4.5.3 Balancing Dataset :

This step aims at addressing class imbalance in the target variable 'quality' by creating a new balanced dataset that contains an equal number of instances (217) for both the 'good quality' and 'bad quality' classes.

CODE:

```
#balancing dataset
good_quality = data[data['quality']==1]
bad_quality= data[data['quality']==0]
bad_quality = bad_quality.sample(frac=1)
bad_quality = bad_quality[:217]
new_df = pd.concat([good_quality,bad_quality])
new_df = new_df.sample(frac=1)
new_df
```

OUTPUT:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
159	6.8	0.600	0.18	1.9	0.079	18.0	86.0	0.99680	3.52	0.57	9.3	1
1190	9.1	0.400	0.57	2.1	0.080	6.0	20.0	0.99652	3.52	0.57	12.5	1
1507	7.5	0.380	0.57	2.3	0.106	5.0	12.0	0.99605	3.52	0.55	11.4	1
540	9.2	0.590	0.24	2.1	0.101	20.0	47.0	0.99880	3.52	0.67	9.6	0
490	9.3	0.775	0.27	2.8	0.078	24.0	56.0	0.99840	3.52	0.67	10.6	1
...	...	...	...	...	...	...	...	...	...	...	...	...
1240	7.5	0.610	0.20	1.7	0.076	36.0	60.0	0.99494	3.52	0.40	9.3	0
1330	7.4	0.600	0.26	2.1	0.083	17.0	91.0	0.99616	3.52	0.56	9.8	1
1317	9.9	0.440	0.46	2.2	0.091	10.0	41.0	0.99638	3.52	0.69	11.9	1
601	7.9	0.460	0.52	2.2	0.071	12.0	35.0	1.00060	3.52	0.56	9.0	1
898	8.3	0.310	0.39	2.4	0.078	17.0	43.0	0.99444	3.52	0.77	12.5	1

1083 rows × 12 columns

Figure-14: Balancing dataset

Again, checking the value count in the new data frame after changing target variable.

CODE:

```
[38] data['quality'].value_counts()
```

```
1    866
0    733
Name: quality, dtype: int64
```

4.5.4 Checking Correlation Between Columns:

Checking correlation between columns involves analysing the relationships and dependencies between pairs of attributes (columns) in your dataset. Correlation analysis helps you understand how changes in one attribute correspond to changes in another, which can be valuable for identifying patterns and potential associations.

CODE:

```
#checking correlation between columns

for a in range(len(data.corr().columns)):
  for b in range(a):
    if abs(data.corr().iloc[a,b])>=0.68:
      name = data.corr().columns[a]
      print(name)
    else:
      print("no 2 features have correlation greater than 0.68")
```

OUTPUT:

```
no 2 features have correlation greater than 0.68
```

Figure-15: Correlation between columns

<u>STEP 4.6: Model Building</u>

Model building is a crucial step in data analysis and machine learning, where we construct predictive models that can generalize patterns from wine dataset to make accurate predictions or classifications on new, unseen data.

4.6.1 Splitting Data

Splitting data in the context of wine quality analysis involves dividing your dataset into subsets for training, validation, and testing to build and evaluate machine learning models.

70% of the data is used for training the model.
30% of the data is used for testing the model.

6.6.1 Normalizing Data

Normalizing the data is an important pre-processing step in wine quality analysis to ensure that the input features have consistent scales and distributions. This process helps improve the performance of machine learning models by preventing certain features from dominating the learning process due to their larger magnitudes.

CODE:

```
#splitting dataset
from sklearn.model_selection import train_test_split
X = new_df.drop('quality',axis=1)
y = new_df['quality']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=101)
#normalizing data
from sklearn.preprocessing import StandardScaler
X_features =X
X=StandardScaler().fit_transform(X)
print(X)
```

OUTPUT:

```
[[-9.00195962e-01  6.33542329e-01 -5.19089347e-01 ... -4.44089210e-16
  -7.13251619e-01 -1.25220436e+00]
 [ 6.81225945e-01 -5.92778212e-01  1.42683918e+00 ... -4.44089210e-16
  -7.13251619e-01  1.83973463e+00]
 [-4.18893643e-01 -7.15410266e-01  1.42683918e+00 ... -4.44089210e-16
  -8.86602495e-01  7.76880601e-01]
 ...
 [ 1.23128574e+00 -3.47514104e-01  8.77987547e-01 ... -4.44089210e-16
   3.26853637e-01  1.25999607e+00]
 [-1.43863746e-01 -2.24882049e-01  1.17736117e+00 ... -4.44089210e-16
  -7.99927057e-01 -1.54207364e+00]
 [ 1.31166151e-01 -1.14462246e+00  5.28718323e-01 ... -4.44089210e-16
   1.02025714e+00  1.83973463e+00]]
```

Figure-16: Normalizing the data

4.6.1 Building Machine Learning Model :

Building a machine learning model using the Random Forest algorithm involves creating an ensemble of decision trees to predict wine quality scores. Random Forest is known for its robustness, accuracy, and ability to handle complex datasets.

CODE:

```
#building model
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(random_state=1)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

STEP 4.7: Model Evaluation

In the model evaluation, we systematically assess the performance of our machine learning model to ensure it's accuracy, reliability, and generalization capability. Model evaluation process is crucial in understanding how well our model is performing in real-world scenarios.

4.7.1 Confusion Matrix

It is a matrix used to determine the performance of the classification models for a given set of test data. It can only be determined if the true values for test data are known. As it shows the errors in the model performance in the form of a matrix, hence also known as an **error matrix.**

CODE:

```
#model evaluation
#confusion matrix
from sklearn import metrics
from sklearn.metrics import accuracy_score
confusion_matrix =metrics.confusion_matrix(y_test,y_pred)
cm=metrics.ConfusionMatrixDisplay(confusion_matrix=confusion_matrix,display_labels=[False,True])
cm.plot()
plt.show()
```

OUTPUT:

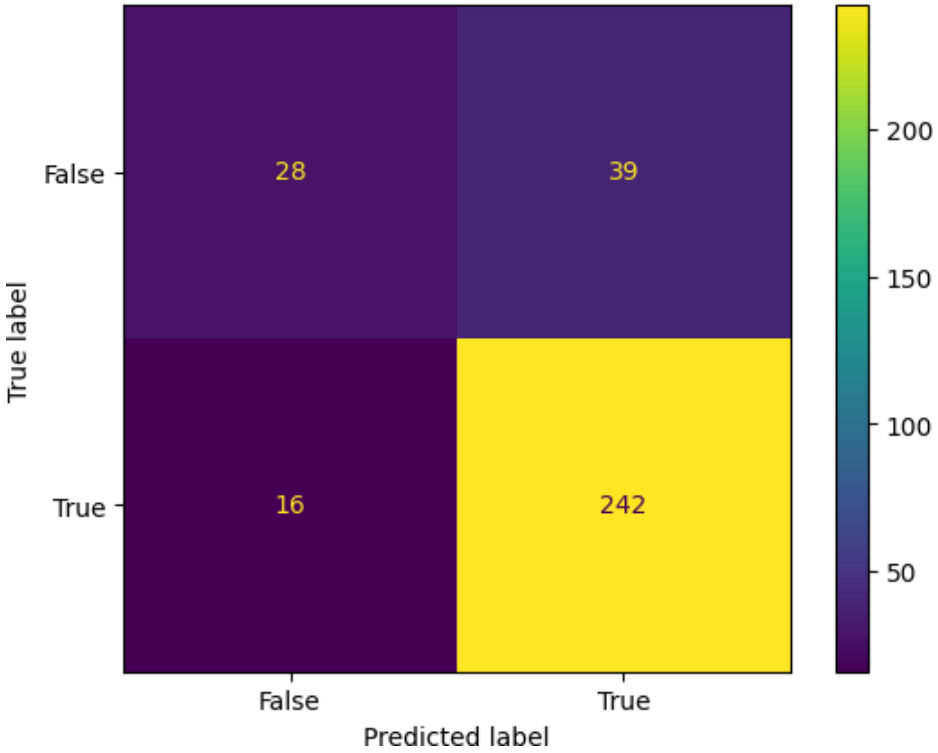


Figure-17: Confusion matrix

4.7.2 Model Accuracy

Model accuracy is a common evaluation metric used to assess the performance of a classification model. It measures the proportion of correctly predicted instances out of the total instances in the dataset.

CODE:

```
#Accuracy
from sklearn.metrics import accuracy_score
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy of the model constructed above is :",accuracy)
```

```
Accuracy of the model constructed above is : 0.8307692307692308
```

Figure-18: Model accuracy

4.7.3 Model Precision

Model precision is a key evaluation metric in classification tasks, particularly when you're focused on the accuracy of positive predictions. It measures the proportion of correctly predicted positive instances out of all instances predicted as positive.

CODE:

```
#Precision
from sklearn.metrics import recall_score
recall = recall_score(y_test,y_pred)
print("Recall is :", recall)
```

```
Recall is : 0.937984496124031
```

Figure-19: Model precision

4.7.4 Internal or Component design structure

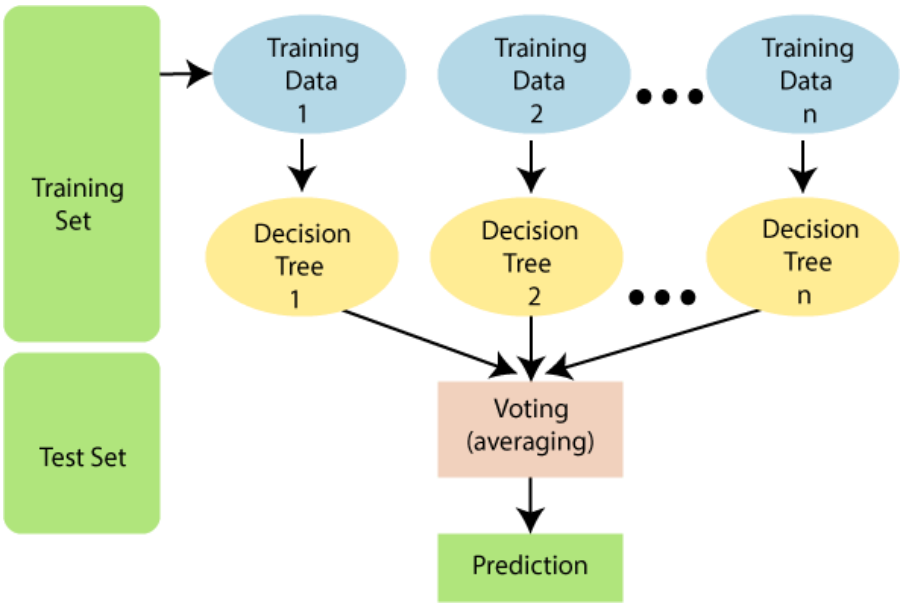


Figure-20: Internal design structure

CHAPTER 5

CONCLUSION

In conclusion, the wine quality analysis project amalgamates data-driven insights with machine learning techniques to decipher the intricate relationship between physicochemical attributes and wine quality. Through robust model building, evaluation, and interpretability, we unveil influential attributes and construct predictive tools. This endeavour bridges the gap between data science and practical winemaking, equipping stakeholders with a nuanced understanding to optimize processes and enhance wine quality. By leveraging advanced methodologies, this project underscores the potential of data analytics to drive informed decision-making in the realm of viticulture, promising impactful strides in quality enhancement and industry advancement.

REFERENCES

- Dastmard, B., 2013. A statistical analysis of the connection between test Results and field claims for ECUs in vehicles.

- Drummond, C., Holte, R.C., 2003. C4.5, Class Imbalance, and Cost Sensitivity: Why Under-sampling beats Over-sampling. Pp. 1–8.

- Er, Y., Atasoy, A., 2016. The Classification of White Wine and Red Wine According to Their Physicochemical Qualities. Int. J. Intell. Syst. Appl. En,305-312. https://doi.org/10.1016/j.procs.2017.12.041

- Hewahi, N.M., Abu Hamra E, 2017. A Hybrid Approach Based on Genetic Algorithm and Particle Swarm Optimization to Improve Neural Network Classification. J. Inf. Technol. Res. JITR 10, 48–68. https://doi.org/10.4018/JITR.2017070104.

- J. Han, Micheline Kamber, Jian Pei, 2012. Data Mining: Concepts and Techniques 3rd Edition. DATA Min. 560.

- Joseph, R., 2018. Grid Search for model tuning [WWW Document] Medium. URLhttps://towardsdatascience.com/grid-search-model-tuning-3319b259367e (accessed 6.6.21) g. 4, 23–26.

- Estabrooks, A., Japkowicz, N., 2001. A Mixture-of-Experts Framework for Learning from Imbalanced Data Sets, in: Hoffmann, F., Hand, D.J.,

- Adams, N., Fisher, D., Guimaraes, G. (Eds.),Advances in Intelligent Data Analysis ,Lecture Notes in Computer Science.

- Gupta, Y., 2018. Selection of important features and predicting wine quality using machine learinng techniques. Procedia comput. Sci.125.